Uniwersytet Gdański

Wydział matematyki, fizyki i informatyki

PRACA NAUKOWA

Maszyna Turinga jako fundament informatyki teoretycznej: złożoność obliczeniowa i granice stosowalności.

Imię i nazwisko autora: Katarzyna Dądera
Nr albumu: 292560
Kierunek: Informatyka ogólnoakademicka

Gdańsk, 2026

Spis treści

1 Wstęp **2**

1.1 Geneza historyczna i problem rozstrzygalności 2

1.2 Budowa i formalna definicja maszyny Turinga 2

1.3 Teza Churcha-Turinga i znaczenie modelu 3

2 Złożoność obliczeniowa i klasy P oraz NP **4**

2.1 Miara zasobów modelu maszyny Turinga 4

2.2 Klasa P 4

2.3 Niedetermizm i klasa NP 5

2.3.1 Problemy NP zupełne 6

2.4 Relacja klas P i NP oraz problem zawierania 7

2.5 Złożoność pamięciowa: klasy L, NL oraz PSPACE 8

2.5.1 Twierdzenie Savitcha i klasy PSPACE 9

3 Granice stosowalności – problemy nierozstrzygalne **10**

3.1 Problem stopu i metoda przekątniowa 10

3.2 Twierdzenie Rice'a i uogólnienie nierozstrzygalności 11

3.3 Konsekwencje dla inżynierii oprogramowania 11

4 Podsumowanie **11**

1 Wstęp

1.1 Geneza historyczna i problem rozstrzygalności

Niezależnie od sformułowania słynnej listy zagadnień matematycznych z przełomu XIX i XX wieku, w 1928 roku niemiecki matematyk David Hilbert zdefiniował tzw. problem decyzyjny „Entscheidungsproblem". Zagadnienie dotyczyło przedstawienia „ogólnego procesu pozwalającego na określenie możliwości udowodnienia dowolnych twierdzeń w logice matematycznej" [1]. Hilbertowi zależało na stworzeniu „procedury mechanicznej", czyli dzisiejszego pojęcia algorytmu, która byłaby w stanie odpowiedzieć na jakikolwiek problem matematyczny z jednoznaczną odpowiedzią w skończonym czasie. Jego ostatecznym celem było ustalenie, czy matematyka jest dziedziną w całości rozstrzygalną. Jako pierwszy przeczącej odpowiedzi na to pytanie udzielił amerykański matematyk Alonzo Church, który w 1936 roku stwierdził, że ogólny przypadek „Entscheidungsproblem" jest nierozwiązywalny. Niemalże w tym samym czasie, pracując zupełnie niezależnie, nieświadomie do tej samej konkluzji doszedł Alan Turing. Był to brytyjski matematyk, który chcąc wykazać ograniczenia matematyki musiał najpierw zdefiniować czym jest proces obliczania. W tym celu stworzył abstrakcyjny model procesu liczenia, znany dziś jako maszyna Turinga.

1.2 Budowa i formalna definicja maszyny Turinga

Była ona w pełni teoretycznym, nieistniejącym fizycznie wytworem matematycznych przemyśleń i rozważań. Poszczególne kroki procesu obliczeniowego maszyny były jedynie zapisywane na papierze w postaci równań i logicznych reguł. Praca maszyny miała za zadanie precyzyjnie naśladować zachowanie człowieka wykonującego konkretne obliczenia. Z racji abstrakcyjnego charakteru maszyny Turinga jej struktura jest niezwykle prosta składająca się jedynie z paru fundamentalnych elementów.

Model opiera się na nieskończonej taśmie podzielonej na komórki, na której zapisywane są dane wejściowe w postaci symboli oraz głowicy odpowiadającej za zapisywanie oraz odczytywanie poruszającej się o jedną pozycję zależnie w prawo lub w lewo. Całkowite działanie głowicy jest oparte na aktualnym stanie wewnętrznym maszyny oraz na tzw. funkcjach przejścia. Funkcje te stanowią zbiór reguł, które określają jaki symbol zapisać oraz w którym kierunku powinna poruszyć się głowica maszyny na podstawie obecnie odczytywanego znaku. Takie zachowanie maszyny można przedstawić w ścisłym zapisie matematycznym.

W podstawowym ujęciu przyjęto model deterministycznej maszyny Turinga, którą formalnie definiuje się za pomocą następujących siedmiu znaków[2]:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}}) \tag{1}$$

gdzie:

- Q - skończony zbiór stanów wewnętrznych, w jakich może znajdować się maszyna,
- Σ - alfabet wejściowy, czyli zbiór symboli, które mogą znajdować się na taśmie na samym początku,
- Γ - alfabet taśmy, który zawiera wszystkie symbole Σ oraz dodatkowo specjalny symbol pusty (ang. *blank symbol*), oznaczający puste miejsce na taśmie,
- δ - funkcja przejścia, które definiuje się jako:

  $$\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$$

  Dla danego stanu i przeczytanego symbolu, funkcja dyktuje nowy stan, symbol do zapisania oraz kierunek ruchu głowicy (L – lewo, R – prawo),
- q_0 - stan początkowy, od którego maszyna zaczyna pracę,
- q_{accept} - stan akceptujący, jeśli maszyna do niego dotrze, obliczenia kończą się sukcesem,
- q_{reject} - stan odrzucający, który oznacza zakończenie pracy z wynikiem negatywnym.

1.3 Teza Churcha-Turinga i znaczenie modelu

Niedługo po stworzeniu maszyny Turinga amerykański matematyk Stephen Cole Kleene przedstawił światu znaczącą do dzisiaj tezę na temat ograniczeń działania maszyn obliczeniowych. Jako pierwszy wyciągnął wniosek na podstawie dwóch niezależnych od siebie badań, formalnie zrównując ich modele. Jak zauważył Kleene: „Teza Turinga, czyli każda funkcja, która w naturalny sposób byłaby uważana za obliczalną jest obliczalna według definicji, to znaczy przez jedną z jego maszyn jest równoważna tezie Churcha"[1]. Postulat Kleene'ego, znany powszechnie jako teza Churcha-Turinga, pozostaje niemożliwy do formalnego udowodnienia. Wynika to z faktu, że stanowi on próbę utożsamienia intuicyjnego, ludzkiego pojęcia skutecznej obliczalności ze ścisłym modelem matematycznym.

Teza ta pokazuje, że maszyna Turinga stanowi nieprzekraczalny limit możliwości urządzeń cyfrowych. Zgodnie z tym postulatem, żaden współczesny ani przyszły klasyczny komputer nie będzie w stanie rozwiązać problemu, z którym maszyna Turinga by sobie nie poradziła. Od momentu jej powstania nie udało się stworzyć modelu o większej mocy obliczeniowej, co umacnia rację Kleene'ego, której aktualnie nie można podważyć.

Nawet współcześnie teza ma wielki wpływ na naukę i jest jednym z najważniejszych teoretycznych podstaw informatyki, co pokazuje pojęcie kompletności Turinga. W dzisiejszej informatyce każdy język programowania lub system obliczeniowy uznaje się za kompletny, jeśli ma taką samą moc obliczeniową co maszyna Turinga przy nieskończonym dostępie do pamięci. Zważając jednak na fizyczną niemożliwość posiadania bezmiernej pamięci w rzeczywistych urządzeniach, w praktyce system uznaje się za kompletny, jeśli jego ograniczenia wynikają jedynie z braku fizycznych zasobów, a nie logiki działania. To rozróżnienie pozwoliło na dalszy rozwój efektywności algorytmów, co stanowi fundament teorii złożoności obliczeniowej.

2 Złożoność obliczeniowa i klasy P oraz NP

2.1 Miara zasobów modelu maszyny Turinga

W zastosowaniach praktycznych wydajność algorytmu mierzy się w sekundach lub milisekundach, jednak w informatyce teoretycznej taka miara jest bezużyteczna, ponieważ opiera się jedynie na parametrach procesora i architektury sprzętu. Właśnie dlatego, aby zbadać złożoność problemów matematycznych w sposób obiektywny wykorzystuje się do tego matematyczny model maszyny Turinga. Wyróżnia się ją na tle innych modeli ze względu na zdefiniowanie złożoności czasowej jako liczby kroków, czyli przejść stanów oraz przesunięć głowicy na taśmie. Maszyna ma za zadanie wykonać obliczenia dla danych wejściowych o rozmiarze n, by zakończyć obliczenia i zatrzymać się w stanie akceptującym lub odrzucającym. W przypadku złożoności pamięciowej określa się ją na podstawie maksymalnej liczby wykorzystanych komórek taśmy podczas całego procesu obliczeniowego.

Z racji tego, że dokładna liczba kroków jest niemożliwa do precyzyjnego ustalenia, analizę tę sprowadza się do najgorszego czasu wykonania. Czas ten zależny jest od liczby danych wejściowych, więc wyrażany jest jako funkcja rozmiaru wejścia n wykorzystając notację dużego $\mathcal{O}$ [3]. Notacja ta pozwala stwierdzić jak szybko rośnie zapotrzebowanie algorytmu na zasoby zwiększając przy tym rozmiar przyjmowanych danych. Dzięki takiemu podejściu do analizy zasobów można wyodrębnić i sklasyfikować problemy decyzyjne w fundamentalne klasy złożoności obliczeniowej.

2.2 Klasa P

W teorii złożoności obliczeniowej pierwszą i najbardziej fundamentalną klasą problemów jest klasa P. Zawiera ona wszystkie problemy decyzyjne, które deterministyczna maszyna Turinga potrafi rozwiązać w czasie wielomianowym. Bazując na wcześniej opisywanej formalnej definicji maszyny deterministycznej można wykazać, że liczba kroków wymagana

do znalezienia rozwiązania jest ograniczona z góry przez funkcję $O(n^k)$. W tym przypadku n to rozmiar danych wejściowych, a k jest stałą liczbą całkowitą większą od zera.

Powszechnie w środowisku informatycznym przyjmuje się zasady tzw. tezy Cobhama-Edmondsa, która utożsamia klasę P z problemami praktycznie obliczalnymi, czyli efektywnie rozwiązywalnymi[4]. Warto zauważyć jednak, że nawet w przypadku czasu o złożoności $O(n^{100})$ z matematycznego punktu widzenia również jest wielomianowy. W praktyce dla dużych zbiorów danych wykonanie takiego przypadku zajęłoby współczesnym komputerom bardzo dużo czasu. Stąd można stwierdzić, że teza jest pewnym uproszczeniem świata rzeczywistego. Mimo to, większość spotykanych w praktyce użytecznych algorytmów z klasy P charakteryzuje się niskimi potęgami, takimi jak $O(n)$, $O(n \log n)$ czy $O(n^2)$, dzięki czemu czas ich wykonania jest błyskawiczny.

Do klasycznych przykładów zagadnień należących do klasy P zalicza się między innymi: podstawowe operacje arytmetyczne, sortowanie zbiorów danych, wyszukiwanie najkrótszej ścieżki w grafie (np. algorytm Dijkstry) czy też wyznaczanie największego wspólnego dzielnika za pomocą algorytmu Euklidesa. Są to zadania, których czas rozwiązywania rośnie w sposób umiarkowany i przewidywalny wraz ze wzrostem rozmiaru danych wejściowych.

2.3 Niedetermizm i klasa NP

Następną, znacznie szerszą i złożoną kategorią problemów w teorii złożoności jest klasa NP. W przypadku tej klasy problemów litera „N" odnosi się do koncepcji niedeterminizmu. Jest to szczególny element odróżniający tę klasę od poprzedniej, ponieważ opiera się na rozszerzonej wersji klasycznego modelu obliczeń o zdolność do podejmowania wielu ścieżek decyzyjnych jednocześnie. W deterministycznej maszynie Turinga, dla każdego stanu i odczytanego symbolu, istnieje dokładnie jedna, z góry określona akcja. Model niedeterministycznej maszyny Turinga nie posiada takiego ograniczenia. Formalnie definiuje się za pomocą analogicznej siódemki uporządkowanej $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$ analogicznie do deterministycznej maszyny Turinga, jednak fundamentalnej zmianie ulega funkcja przejścia δ. W modelu niedeterministycznym maszyna może mieć kilka dozwolonych ruchów dla tej samej konfiguracji, co w zapisie matematycznym realizuje się poprzez mapowanie w zbiór potęgowy P[2] (zbiór wszystkich podzbiorów):

$$\delta : Q \times \Gamma \to \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

Oznacza to, że maszyna potrafi w każdym kroku wybrać właściwą ścieżkę lub w procesie obliczeniowym tworzy się drzewo rozgałęziających się możliwości, które są sprawdzane równolegle. Jeśli choć jedna z gałęzi tego drzewa doprowadzi maszynę do stanu akceptującego q_{accept} w czasie wielomianowym można stwierdzić, że maszyna akceptuje dane wejściowe.

Klasa NP to zatem zbiór wszystkich problemów decyzyjnych, które mogą zostać rozwiązane przez niedeterministyczną maszynę Turinga w czasie wielomianowym $O(n^k)$. Jednak z racji tego, że jest to abstrakcyjny konstrukt, którego nie da się zrealizować w formie fizycznego procesora w praktyce informatycy stosują równoważną definicję bazującą na pojęciu weryfikatora. Zgodnie z nią, problem należy do klasy NP, jeśli każde prawidłowe rozwiązanie można zweryfikować pod kątem poprawności przez zwykłą maszynę deterministyczną w czasie wielomianowym [2]. Formalnie twierdzenie to zapisuje się w następujący sposób: Język, czyli problem decyzyjny L jest akceptowany przez niedeterministyczną maszynę Turinga M w czasie wielomianowym wtedy i tylko wtedy, gdy istnieje wielomian p oraz taka deterministyczna maszyna Turinga M_R działająca w czasie wielomianowym, że język L można przedstawić w postaci:

$$L = \{w \mid \exists y, |y| \leq p(|w|) \text{ oraz } w\#y \text{ jest akceptowane przez } M_R\}$$

Zapis ten precyzyjnie obrazuje koncepcję weryfikacji. Dla dowolnego słowa wejściowego w, słowo y pełni rolę certyfikatu, a M_R jest deterministyczną maszyną weryfikującą jego poprawność. Warunek ograniczający $|y| \leq p(|w|)$ gwarantuje, że certyfikat ma długość wielomianową, dzięki czemu sama jego analiza przez maszynę M_R nie przekroczy czasu wielomianowego. Twierdzenie to wykazuje, że problemy z klasy NP to w zagadnienia posiadające krótkie, wielomianowe certyfikaty oraz szybkie, deterministyczne algorytmy do ich sprawdzania.

Przykładem zagadnienia należącego do klasy NP, który pokazuje koncepcję weryfikacji i certyfikatu jest decyzyjna wersja problemu komiwojażera (ang. Traveling Salesman Problem, TSP). Wejściem dla tego problemu jest graf z całkowitoliczbowymi wagami na krawędziach reprezentującymi odległości między miastami oraz limit wagi W[5]. Pytanie decyzyjne brzmi: czy w danym grafie istnieje cykl Hamiltona (trasa odwiedzająca każdy węzeł dokładnie raz i wracająca do punktu wyjścia), którego całkowita waga nie przekracza wartości W? Znalezienie takiego cyklu jest niezwykle trudne, ponieważ wymaga przeszukania liczby kombinacji rosnącej w tempie silni. Jednakże w przypadku gotowej propozycji trasy pełniącą rolę certyfikatu y, deterministyczna maszyna Turinga potrafi w czasie wielomianowym zweryfikować jej poprawność. Jedynie wystarczy sprawdzić, czy podana sekwencja faktycznie jest cyklem Hamiltona i czy suma wag jej krawędzi jest mniejsza lub równa W.

2.3.1 Problemy NP zupełne

W obrębie klasy NP istnieje szczególna oraz istotna podgrupa zagadnień, które uważa się jako najtrudniejsze ze wszystkich problemów w tej klasie. Są to problemy NP zupełne. Mówimy, że język, czyli przykładowy problem L_1 jest wielomianowo redukowalny do języka L_2 (co zapisuje się jako $L_1 \leq_p L_2$), jeśli istnieje wielomianowo obliczalna funkcja,

która potrafi przekształcić każde wejście problemu L_1 w wejście problemu L_2, zachowując przy tym wynik decyzyjny, czyli odpowiedź „tak" lub „nie".

Problem L nazywamy NP-zupełnym, jeśli spełnia on jednocześnie dwa warunki:

- Należy do klasy NP ($L \in NP$)
- każdy inny problem należący do klasy NP można do niego zredukować w czasie wielomianowym [3].

Pojęcie to po raz pierwszy sformułował Stephen Cook w 1971 roku (niezależnie od niego dokonał tego także Leonid Levin). W swoim fundamentalnym twierdzeniu, znanym dziś jako twierdzenie Cooka-Levina, udowodnili oni istnienie pierwszego problemu NP-zupełnego, którym był problem spełnialności formuł logicznych (SAT).

Klasycznym i niezwykle ważnym wariantem tego zagadnienia jest problem 3-SAT [3]. W problemie tym wejście stanowi formuła logiczna zapisana w koniunkcyjnej postaci normalnej (CNF), w której każda klauzula (nawias) zawiera dokładnie trzy literały połączone alternatywą, np. $(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_4 \vee \neg x_5)$. Pytanie decyzyjne brzmi: czy istnieje takie przypisanie wartości logicznych zmiennym, dla którego cała formuła przyjmuje wartość prawdy? Problem 3-SAT ma fundamentalne znaczenie, ponieważ ze względu na jego regularną strukturę, znacznie łatwiej jest projektować funkcje redukujące z 3-SAT do innych, nowych problemów informatycznych, dowodząc tym samym ich NP-zupełności.

Wyjątkowość problemów NP-zupełnych polega na tym, że są one ze sobą głęboko powiązane. Gdyby udało się znaleźć deterministyczny algorytm rozwiązujący jakikolwiek jeden problem NP-zupełny w czasie wielomianowym, dzięki mechanizmowi redukcji oznaczałoby to automatyczne znalezienie szybkiego rozwiązania dla wszystkich problemów w całej klasie NP.

2.4 Relacja klas P i NP oraz problem zawierania

Z przytoczonych wcześniej definicji wyłania się bezpośrednia relacja między omawianymi klasami. Ze względu na to, że każda deterministyczna maszyna Turinga jest w istocie szczególnym przypadkiem maszyny niedeterministycznej (czyli takim, w którym zbiór dozwolonych przejść zawsze ogranicza się do jednej ścieżki) wynika z tego, że każdy problem rozwiązywalny w czasie wielomianowym można również w takim czasie zweryfikować. Fakt ten w teorii złożoności zapisuje się za pomocą relacji zawierania zbiorów:

$$P \subseteq NP$$

Najważniejszym i wciąż nierozstrzygniętym pytaniem współczesnej informatyki teoretycznej oraz matematyki pozostaje kwestia charakteru tego zawierania. Pytanie to, sformułowane po raz pierwszy w 1971 roku przez Stephena Cooka, brzmi: czy klasa P jest

właściwym podzbiorem klasy NP ($P \neq NP$), czy też w rzeczywistości obie te klasy są sobie równe ($P = NP$)?

Gdyby okazało się, że $P = NP$, oznaczałoby to, że dla każdego problemu, którego rozwiązanie można w czasie wielomianowym zweryfikować, istnieje również ukryty, szybki algorytm pozwalający na jego znalezienie. Taki obrót spraw miałby katastrofalne skutki dla współczesnego bezpieczeństwa cyfrowego. Cała współczesna kryptografia opiera się na założeniu, że faktoryzacja dużych liczb pierwszych jest problemem trudnym do rozwiązania, wymagającym czasu wykładniczego, ale niezwykle łatwym do sprawdzenia. Równość $P = NP$ zniszczyłaby fundamenty tego bezpieczeństwa.

Z tego względu zdecydowana większość środowiska naukowego skłania się ku hipotezie, że $P \neq NP$, co oznacza, że istnieją problemy (takie jak problemy NP-zupełne), dla których z natury nie istnieje i nigdy nie powstanie wielomianowy algorytm je rozwiązujący. Waga tego problemu dla nauki i ludzkości jest tak ogromna, że w 2000 roku Instytut Matematyczny Claya wpisał go na listę siedmiu Problemów Milenijnych, oferując nagrodę w wysokości miliona dolarów za jego formalne udowodnienie lub obalenie [2].

2.5 Złożoność pamięciowa: klasy L, NL oraz PSPACE

Dla klas P oraz NP jednym z istotnych parametrów ograniczających możliwości obliczeniowe jest miara czasu. Jednakże drugim, równie istotnym parametrem decydującym o efektywności maszyny Turinga, jest pamięć, czyli maksymalna liczba komórek taśmy wykorzystana podczas całego procesu rozwiązywania problemu.

Złożoność pamięciową definiuje się formalnie: maszyna Turinga działa w pamięci ograniczonej przez funkcję $S(n)$, jeżeli dla każdego słowa wejściowego w, żadne obliczenie maszyny nie odwiedza więcej niż $S(|w|)$ komórek na jej taśmach roboczych[4]. Zakłada się, że raz odwiedzona komórka taśmy roboczej nie staje się z powrotem pusta. Na tej podstawie, analogicznie do klas czasowych, wyodrębnia się dwie główne rodziny klas przestrzennych:

- $DSPACE(S(n))$ - zbiór języków, inaczej problemów decyzyjnych, które mogą zostać rozwiązane przez deterministyczną maszynę Turinga przy użyciu pamięci rzędu $O(S(n))$, gdzie n to rozmiar wejścia.
- $NSPACE(S(n))$ - zbiór języków rozwiązywalnych przez niedeterministyczną maszynę Turinga z analogicznym ograniczeniem pamięciowym rzędu $O(S(n))$.

Szczególną rolę w analizie złożoności pamięciowej odgrywają problemy, które wymagają niezwykle małej ilości miejsca, to znaczy mniejszej nawet niż sam rozmiar początkowych danych wejściowych. Aby poddać je badaniu, w informatyce teoretycznej stosuje się zmodyfikowany model maszyny Turinga, w którym wydziela się taśmę wejściową (dostępną wyłącznie do odczytu) oraz osobną taśmę roboczą. Pozwala to na wyodrębnienie dwóch kluczowych klas logarytmicznych:

- L - klasa problemów rozwiązywalnych deterministycznie w pamięci logarytmicznej: $L = DSPACE(\log n)$.
- NL - klasa problemów rozwiązywalnych niedeterministycznie w pamięci logarytmicznej: $NL = NSPACE(\log n)$.

Klasycznym zagadnieniem ilustrującym klasę L jest problem weryfikacji palindromu. Deterministyczna maszyna Turinga rozwiązująca ten problem nie musi kopiować całego słowa wejściowego. Wystarczy, że na taśmie roboczej będzie przechowywać jedynie dwa wskaźniki. Pierwszy z nich wskazujący na początek, a drugi na koniec badanego słowa, gdzie podczas stopniowego zbliżania do siebie można porównać odczytywane znaki. Ponieważ zapisanie indeksu dla słowa o długości n wymaga przestrzeni rzędu $\log n$ bitów, algorytm ten mieści się w klasie pamięci logarytmicznej.

Z kolei ważnym przykładem obrazującym klasę NL jest problem osiągalności w grafie skierowanym (odpowiedź na pytanie, czy z wierzchołka s istnieje ścieżka do wierzchołka t). Niedeterministyczna maszyna Turinga rozwiązująca ten problem musi jedynie „zgadywać" kolejny wierzchołek na trasie i pamiętać na taśmie roboczej wyłącznie indeks aktualnego węzła. Ponieważ zapisanie indeksu wierzchołka również wymaga jedynie $\log n$ bitów, maszyna mieści się w pamięci logarytmicznej względem rozmiaru całego grafu.

2.5.1 Twierdzenie Savitcha i klasy PSPACE

Analiza złożoności przestrzennej ujawnia fundamentalne zjawisko matematyczne, które drastycznie odróżnia naturę pamięci od natury czasu. Jak dowodzi sformułowane w 1970 roku twierdzenie Savitcha [6], dla dowolnej funkcji ograniczającej $S(n) \geq \log n$ zachodzi następująca relacja:

$$NSPACE(S(n)) \subseteq DSPACE((S(n))^2)$$

Twierdzenie to wykazuje, że każdą niedeterministyczną maszynę ograniczoną pamięciowo można zawsze zasymulować maszyną deterministyczną, co najwyżej podnosząc ilość wymaganej taśmy do kwadratu. Z faktu tego wynika potężny wniosek: w przypadku limitów wielomianowych, klasy deterministyczne i niedeterministyczne ulegają zrównaniu ($PSPACE = NPSPACE$). Zjawisko to obnaża fundamentalną właściwość sprzętu obliczeniowego: pamięć operacyjna może być nadpisywana i wykorzystywana wielokrotnie do przeglądania alternatywnych ścieżek decyzyjnych, podczas gdy upływ czasu pozostaje procesem całkowicie nieodwracalnym.

3 Granice stosowalności – problemy nierozstrzygalne

Wcześniejsza analiza klas złożoności P, NP oraz PSPACE zakładała, że badane problemy są algorytmicznie rozwiązywalne, a główną barierą obliczeniową jest jedynie brak odpowiednio wydajnych zasobów. Teoria obliczeń definiuje jednak znacznie potężniejsze ograniczenie. Odpowiadając na słynny problem decyzyjny Hilberta (Entscheidungsproblem), Alan Turing wykazał matematycznie, że istnieją zagadnienia, na które nie potrafi odpowiedzieć żadna maszyna licząca. Żaden klasyczny komputer, nawet dysponujący nieskończonym czasem i pamięcią, nie jest w stanie podać dla nich jednoznacznego wyniku. Klasę tego typu zagadnień określa się jako problemy nierozstrzygalne

3.1 Problem stopu i metoda przekątniowa

Najsłynniejszym i najbardziej fundamentalnym problemem nierozstrzygalnym jest problem stopu. Polega on na postawieniu następującego pytania[2]: „Czy można stworzyć uniwersalny algorytm, który na podstawie kodu dowolnego innego programu (maszyny Turinga) oraz przypisanych mu danych wejściowych, jednoznacznie i w skończonym czasie określi, czy program ten kiedykolwiek zakończy swoje działanie, czy też wpadnie w nieskończoną pętlę?".

Zrozumienie tego problemu wymaga odwołania się do koncepcji Uniwersalnej Maszyny Turinga, która potrafi symulować działanie dowolnej innej maszyny, traktując jej opis jako dane wejściowe. Wykorzystując ten mechanizm, Alan Turing udowodnił matematyczną niemożliwość rozwiązania problemu stopu, stosując metodę sprowadzenia do sprzeczności, inspirowaną przekątniowym dowodem Cantora [2]. Rozumowanie to przebiega następująco: załóżmy, że istnieje hipotetyczna, deterministyczna maszyna Turinga H, która potrafi rozstrzygnąć problem stopu. Maszyna H przyjmuje na wejściu opis programu P oraz jego dane x. Jeśli P zatrzymuje się dla danych x, maszyna H przechodzi do stanu akceptującego; w przeciwnym razie przechodzi do stanu odrzucającego. Mając taką maszynę, możemy skonstruować nową maszynę paradoksalną D, która wykorzystuje H jako swój wewnętrzny podprogram. Maszyna D przyjmuje na wejściu kod dowolnego programu P i pyta maszynę H, co zrobiłby program P, gdyby otrzymał jako dane wejściowe swój własny kod. Następnie maszyna D celowo wykonuje akcję odwrotną do werdyktu H. Jeśli H orzeknie, że program się zatrzyma, maszyna D wchodzi w nieskończoną pętlę. Natomiast, gdy H orzeknie, że program wpadnie w pętlę, maszyna D natychmiast kończy pracę.

Paradoks pojawia się w momencie, w którym uruchomimy maszynę D, podając jej na wejściu jej własny kod (uruchomienie $D(D)$). Jeśli D ma się zatrzymać, z jej definicji wynika, że musi wejść w nieskończoną pętlę. Jeśli z kolei ma wpaść w pętlę, musi się zatrzymać. Ta nierozwiązywalna sprzeczność logiczna jest ostatecznym matematycznym dowodem na to, że pierwotne założenie było błędne. maszyna H jako uniwersalny analizator problemu stopu, nie może istnieć.

3.2 Twierdzenie Rice'a i uogólnienie nierozstrzygalności

Problem stopu nie jest jedynie odosobnioną anomalią matematyczną. W 1953 roku Henry Gordon Rice udowodnił twierdzenie, które drastycznie poszerza granice nierozstrzygalności. Twierdzenie Rice'a głosi, że każda nietrywialna własność semantyczna języka rozpoznawanego przez maszynę Turinga jest nierozstrzygalna [5]. Własność semantyczna dotyczy tego, co program robi (jakie dane akceptuje), a nie tego, jak jest napisany. Własność nietrywialna to z kolei taka, którą posiadają niektóre programy, ale nie wszystkie.

Z twierdzenia tego wynika bezpośrednio, że niemożliwe jest stworzenie ogólnego algorytmu, który potrafiłby na podstawie samego kodu źródłowego odpowiedzieć na pytania dotyczące docelowego zachowania programu, takie jak charakter generowanych wyników wyjściowych czy ostateczne zapotrzebowanie na zasoby pamięciowe.

3.3 Konsekwencje dla inżynierii oprogramowania

Z pozoru teoretyczne rozważania Turinga i Rice'a stanowią dziś absolutny fundament i wyznaczają granice stosowalności we współczesnej inżynierii oprogramowania. Matematyczne dowody nierozstrzygalności gwarantują, że:

- Nigdy nie powstanie idealny kompilator: Narzędzia programistyczne i środowiska (IDE) nigdy nie będą w stanie przed uruchomieniem wyłapać wszystkich błędów, nieskończonych pętli czy wycieków pamięci w dowolnym kodzie.
- Nigdy nie powstanie perfekcyjny program antywirusowy: Ponieważ określenie, czy dany plik po uruchomieniu wykona niepożądaną akcję, jest badaniem nietrywialnej właściwości semantycznej programu, problem ten z mocy twierdzenia Rice'a jest nierozstrzygalny. Programy antywirusowe mogą jedynie bazować na heurystykach, statystykach i sygnaturach, ale nie mogą zaoferować absolutnej, matematycznej gwarancji bezpieczeństwa dla każdego nowo napisanego kodu.

Granice wyznaczone przez model maszyny Turinga uświadamiają nam zatem, że informatyka, mimo swojego ogromnego potencjału i zawrotnego rozwoju technologicznego, pozostaje dziedziną ograniczoną przez nienaruszalne prawa logiki.

4 Podsumowanie

Rozwój informatyki teoretycznej nierozerwalnie wiąże się z abstrakcyjnym modelem maszyny Turinga, który stanowi fundament definiujący istotę procesów obliczeniowych. To z

pozoru proste urządzenie matematyczne nie tylko pozwala na precyzyjną klasyfikację problemów decyzyjnych w oparciu o ich złożoność czasową i pamięciową, ale również uświadamia fundamentalne różnice między łatwością weryfikacji, a skrajną trudnością poszukiwania rozwiązań. Nierozstrzygnięty dotąd dylemat dotyczący relacji klas P i NP pokazuje teoretyczną słabość współczesnych systemów kryptograficznych, na których opiera się dzisiejsze cyfrowe bezpieczeństwo. Ponadto, matematyczne dowody nierozstrzygalności, takie jak problem stopu czy twierdzenie Rice'a, wyznaczają twarde, nieprzekraczalne bariery inżynierii oprogramowania. Wynika z nich bezpośrednio, że stworzenie narzędzi idealnych, takich jak bezbłędne kompilatory czy stuprocentowo skuteczne programy antywirusowe, jest zjawiskiem logicznie niemożliwym. Ostatecznie uświadamia to, że pomimo zawrotnego postępu technologicznego i stałego wzrostu mocy procesorów, możliwości wszystkich komputerów zawsze pozostaną zależne od nienaruszalnych praw matematyki i logiki.

Bibliografia

[1] Charles Petzold, *The Annotated Turing: A Guided Tour Through Alan Turing's Historic Paper on Computability and the Turing Machine*, 2008 by Wiley Publishing, Inc, Indianapolis, Indiana

[2] Michael Sipser, *Introduction to the Theory of Computation, Third Edition*, 2012 Cengage Learning

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms, Third Edition*, 2009 Massachusetts Institute of Technology

[4] Sanjeev Arora and Boaz Barak, *Computational Complexity: A Modern Approach* 2009 Cambridge University Press

[5] J. E. Hopcroft, R. Motwani, J. D. Ullman, *Introduction to Automata Theory, Languages and Computation, Second Edition* 2001 Addison-Wesley

[6] Christos H. Papadimitriou *Złożoność obliczeniowa* 2007 Wydawnictwa Naukowo-Techniczne